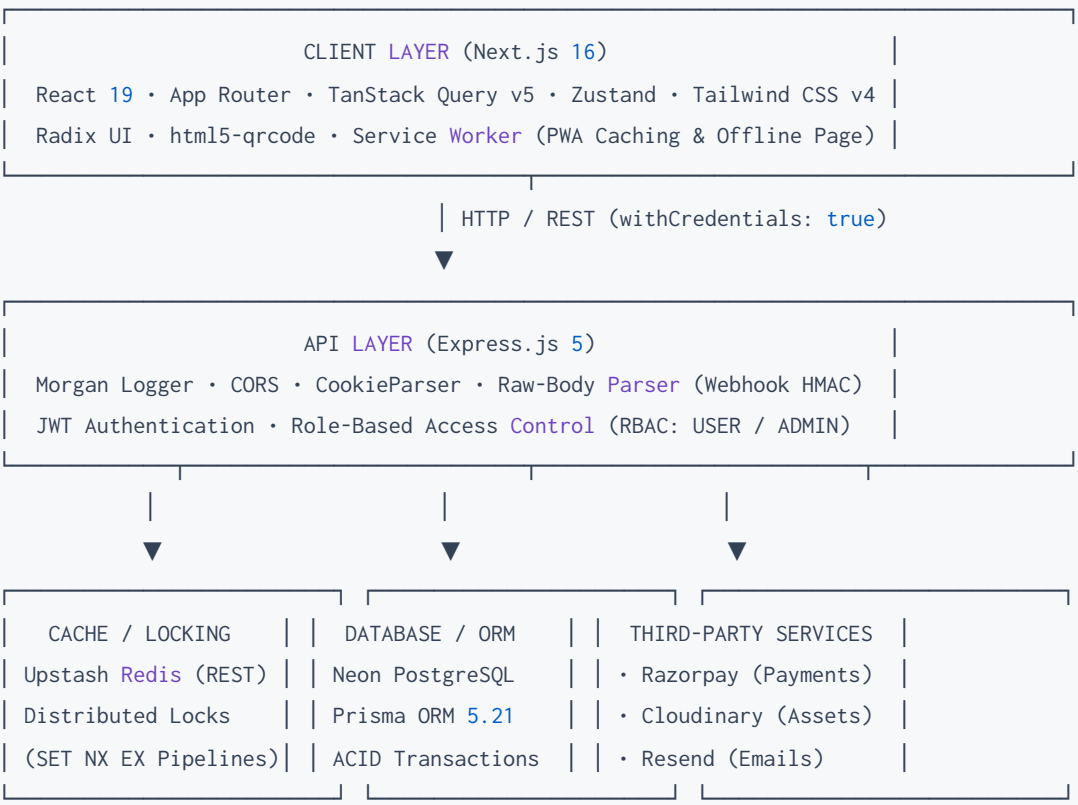
InstantBook: Comprehensive Project & Technical Defense Guide

1. Executive Summary (The 30-Second Elevator Pitch)

> **InstantBook** is a high-concurrency event ticketing platform and Progressive Web App (PWA) built with **Next.js 16**, **Express 5**, **PostgreSQL (Prisma ORM)**, and **Upstash Redis**. It eliminates ticket scalping and double-booking race conditions through a **two-phase distributed locking mechanism** (in-memory Redis lock followed by an ACID database transaction). The system features an interactive 2D seat matrix, PCI-compliant payment processing via **Razorpay**, automated transactional emails with **Resend**, dynamic QR code ticket generation stored on **Cloudinary**, and an **atomic, anti-duplicate venue check-in scanner** powered by `html5-qrcode`.

2. Full Technology Stack & Architectural Breakdown



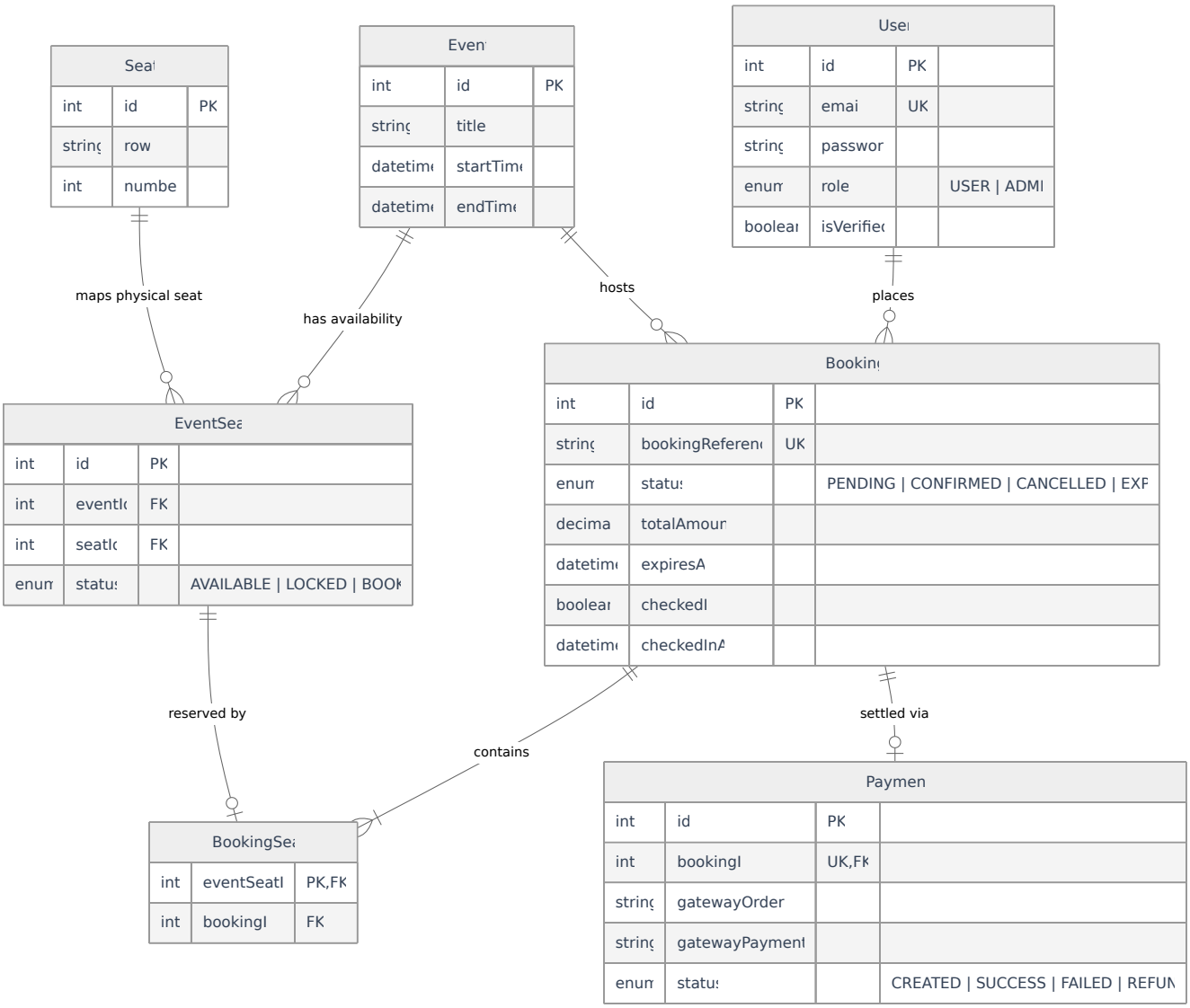
Layer	Technologies Used	Exact Purpose
Frontend Framework	**Next.js 16.1 (React 19)**	App Router, SSR/CSR, route grouping, client components (`use client`).

Layer	Technologies Used	Exact Purpose
State Management	**Zustand 5**	Lightweight client-side authentication state (`user`, `setUser`, `logout`).
Data Fetching	**TanStack React Query v5**	Server-state caching, automatic background refetching, query invalidation.
Styling & UI	**Tailwind CSS v4**, Radix UI, Lucide Icons, Sonner	Glassmorphic dark UI, accessible dialog modals, toast notifications.
Mobile / PWA	`public/sw.js`, `manifest.json`	Service Worker with network-first navigation fallback to `/offline`, cache-first for static assets.
Scanner	`html5-qrcode`	Hardware camera stream integration for venue ticket scanning.
Backend Framework	**Node.js 24 + Express 5**	RESTful routing, custom middleware pipelines, ES Modules (`import/export`).
ORM & Database	**Prisma 5.21 + PostgreSQL**	Type-safe queries, migration control, `@prisma/adapter-pg` pooler support.
Cache & Concurrency	**Upstash Redis**	Distributed in-memory seat locks (`@upstash/redis` REST client).
Payment Gateway	**Razorpay Node SDK**	Order creation, payment capture, automated refunds, HMAC SHA-256 webhook verification.
Media & Storage	**Cloudinary** (`multer-storage-cloudinary`)	Event banner hosting and dynamic QR PNG generation storage.
Email Delivery	**Resend**	Transactional HTML email dispatch with ticket details and embedded QR codes.
Task Scheduling	**node-cron**	Background cron job running every minute (`* * * * *`) to purge expired bookings.

---

3. Database Schema & Data Modeling

The relational model separates **physical venue topology** from **event-specific availability**:



Critical Database Design Highlights:

1. **Separation of `Seat` vs `EventSeat`:**
   - `Seat`: The physical chair in the auditorium (e.g., Row A, Number 5). `@@unique([row, number])`.
   - `EventSeat`: State of that chair for a *specific event* (`AVAILABLE`, `LOCKED`, `BOOKED`). Compound unique index `@@unique([eventId, seatId])` and query optimization index `@@index([eventId, status])`.
2. **Strict Concurrency Guard via `BookingSeat`:**
   - `BookingSeat.eventSeatId` is the primary key (`@id`).
   - **Why this matters**: In relational terms, a specific `EventSeat` row can **never** be linked to more than one booking simultaneously. Any duplicate insert immediately fails at the database engine level with a unique constraint violation.

---

4. Deep-Dive: Core Technical Workflows

A. The Two-Tier Concurrency & Distributed Locking Mechanism

When thousands of users click on the same seat at the same second:

```
[User clicks 'Book Now']
        |
        ▼
[Phase 1: In-Memory Redis Lock]
  • Multi-key Redis Pipeline (`redis.multi()`)
  • For each seat: `SET lock:<eventId>:<seatId> <userId> NX EX 120`
  • If ANY seat fails:
      ↳ Rollback: Execute cleanup pipeline (`DEL lock:...`)
      ↳ Throw 409 Conflict: "One or more seats already locked"
        | (All locks acquired)
        ▼
[Phase 2: ACID Database Transaction (`prisma.$transaction`)]
  • Purge stale `BookingSeat` rows from previous EXPIRED/CANCELLED holds.
  • Check DB availability: `SELECT WHERE id IN (...) AND status = 'AVAILABLE'`.
  • Create `Booking` with status `PENDING` and `expiresAt = now + 5 minutes`.
  • Create `BookingSeat` join rows.
  • Update `EventSeat.status` to `LOCKED`.
        |
        ▼
[Frontend: Open Razorpay Modal with 5-minute checkout countdown]
```

Why use both Redis and PostgreSQL?

- **Redis (Tier 1):** Ultra-low latency (~2ms). Blocks 99% of race collisions in memory before they hit PostgreSQL, preventing connection pool exhaustion on Neon DB.
- **PostgreSQL (Tier 2):** ACID durability. Even if Redis restarts, the database enforces row integrity via foreign keys and the `BookingSeat` primary key constraint.

B. Payment Webhook Verification & Idempotency

Payment verification does **not** rely on the frontend redirect (which can be blocked, spoofed, or closed by the user). It relies on **server-to-server Razorpay webhooks**:

1. **Raw Body Capturing**: In app.js:

   ```
   app.use("/api/payments/webhook", express.raw({ type: () => true }));
   ```

   Why? Standard JSON body parsers alter whitespace and key ordering. Cryptographic HMAC verification **fails** unless executed against the exact raw byte stream sent over the wire.
2. **HMAC SHA-256 Signature Verification**:

   ```
   const expectedSignature = crypto
     .createHmac("sha256", process.env.RAZORPAY_WEBHOOK_SECRET)
     .update(rawBody)
     .digest("hex");
   return expectedSignature === req.headers["x-razorpay-signature"];
   ```

3. **Idempotency Guard**:

```
if (booking.status !== "PENDING") return null;
```

   If Razorpay re-delivers the webhook event (standard behavior under network hiccups), the service detects that the booking is no longer `PENDING` and exits immediately without executing double ticket generation or double email dispatch.
4. **Finalization (`finalizeBooking`)**:
   - `EventSeat.status` flips from `LOCKED` → `BOOKED`.
   - `Booking.status` flips from `PENDING` → `CONFIRMED`.
   - `Payment.status` flips to `SUCCESS`.
   - Redis locks are deleted (`redis.del`).
   - Cloudinary generates and uploads the QR ticket.
   - Resend delivers the confirmation email.

C. Automated Expiration & Self-Healing Sweeper

What happens if a user locks seats but closes the browser or abandons checkout?

1. **The Background Sweeper (expireBookings.job.js)**:
   - Powered by `node-cron`, runs every minute (`* * * * *`).
   - Query: `SELECT FROM Booking WHERE status = 'PENDING' AND expiresAt < NOW()`.
   - For each expired booking, executes an atomic transaction:
     1. Deletes `BookingSeat` rows.
     2. Sets `EventSeat.status = 'AVAILABLE'`.
     3. Marks `Booking.status = 'EXPIRED'`.
2. **On-the-Fly Self-Healing**:
   - If a new user attempts to book a seat that has an expired hold before the cron runs, the `createBooking` service cleans up stale expired records in the database on-demand before acquiring the new lock.

D. QR Code Lifecycle & Venue Anti-Duplicate Check-in

1. **Payload Generation**: The QR payload encodes a domain-namespaced booking reference:

```
INSTANT-BOOK:BOOK-7c9e6679-7425-40de-944b-e07fc1f90ae7
```

   Generated using `qrcode` with **Error Correction Level `H` (High)**: allows up to 30% damage or poor camera lighting while still scanning reliably. Uploaded to Cloudinary under `instant-book/tickets/booking_<id>`.
2. **Atomic Compare-and-Swap (CAS) Check-In (ticket.controller.js)**: To prevent two gatekeepers at two different venue turnstiles from admitting the same ticket simultaneously:

```
const updateResult = await prisma.booking.updateMany({
  where: {
```

```
    id: booking.id,
    checkedIn: false,     // Atomic condition
    status: "CONFIRMED",
  },
  data: {
    checkedIn: true,
    checkedInAt: new Date(),
  },
});
```

- If `updateResult.count === 1`: Check-in is successful.
- If `updateResult.count === 0`: A concurrent scan has already claimed the ticket. The API rejects entry and returns: `"Duplicate check-in detected! This ticket was already checked in at <formattedTime>."`

3. **Admin Check-in Override (resetCheckIn)**: Staff can reset a check-in via `/api/tickets/reset-checkin` if a patron accidentally scanned before entering.

E. Refund & Cancellation Pipeline

- **User-Initiated Cancellation**:
  - Accessible via `/bookings/[id]`.
  - Condition: Permitted **only if** `status === 'CONFIRMED'` **and** `checkedIn === false`. Once a patron enters the venue, cancellation is forbidden.
- **Automated Gateway Refund**:
  - Express calls `razorpay.payments.refund(payment.gatewayPaymentId, { amount: totalAmount * 100 })`.
  - Upon success, marks `Payment.status = 'REFUNDED'`, marks `Booking.status = 'CANCELLED'`, and restores `EventSeat.status = 'AVAILABLE'`.

F. Security & Session Architecture

1. **Cross-Origin HTTP-Only Cookies**:
   - JWT tokens are stored in `res.cookie('token', token, { httpOnly: true, secure: true, sameSite: 'none', maxAge: 7 days })`.
   - **Why `httpOnly`?** JavaScript running in the browser cannot read `document.cookie`, completely neutralizing Cross-Site Scripting (XSS) token theft.
   - **Why `sameSite: "none"` + `secure: true`?** Because the frontend is hosted on Vercel (`*.vercel.app`) while the backend may reside on a separate API subdomain. Cross-site cookie transmission requires modern SameSite relaxation paired with mandatory HTTPS.
2. **Role-Based Access Control (RBAC)**:
   - `protect`: Verifies JWT expiration (`TokenExpiredError`) and attaches `req.user`.
   - `restrictTo("ADMIN")`: Blocks standard users from accessing `/api/events/create`, `/api/tickets/verify`, or triggering refunds.

---

5. Glossary of Technical Terms to Use in Viva/Interviews

Term	How to Explain It in the Interview
Optimistic Concurrency Control	Handling high traffic without heavy database table locks by validating conditions at update time (e.g., `where: { checkedIn: false }` or Redis `SET NX`).
Distributed Lock	A synchronization mechanism across independent server instances using an external coordinator (Upstash Redis) to ensure mutually exclusive access to shared resources.
Pipeline (Redis Multi/Exec)	Batching multiple Redis commands into a single round-trip network request to achieve atomic execution and reduce latency.
ACID Transaction	Atomicity, Consistency, Isolation, Durability. Guaranteed in Prisma via `prisma.$transaction`, ensuring either all database updates succeed or none do.
HMAC SHA-256	Hash-Based Message Authentication Code. Uses a shared secret to calculate a cryptographic hash of raw request bytes, proving the payload originated from Razorpay and was not tampered with in transit.
Idempotency	The property of an operation where executing it multiple times produces the exact same result as executing it once (crucial for webhook handling).
PWA (Progressive Web App)	Web applications that leverage browser Service Workers, cache manifests, and app manifests to provide offline capability, installability, and fast caching like native apps.
Atomic Compare-and-Swap (CAS)	A concurrency primitive that updates a record only if its current state matches an expected value (used to stop duplicate QR check-ins).
TTL (Time-To-Live)	An expiration duration set on Redis keys (`EX 120`) and bookings (`expiresAt: 5 mins`) that automatically purges stale data.
Multi-Stage Dockerfile	Splitting a container build into a `builder` stage (with compilers/devDeps) and a minimal `production` stage (only runtime files) to keep image sizes small and secure.

---

6. Top 15 Most Anticipated Questions & Model Answers

Q1: "How does InstantBook prevent two people from buying the same seat at the exact same moment?"

> **Model Answer:**
> "We implement a **two-tier defense mechanism**. First, at the memory layer, we use **Upstash Redis** with an atomic pipeline. We execute `SET lock:<eventId>:<seatId> <userId> NX EX 120`. The `NX` flag

> ensures the key is only created if it doesn't already exist. If any seat in a user's selection fails, the pipeline immediately rolls back all acquired keys and returns an error.
> Second, at the database layer inside a `prisma.$transaction`, the selected seats must have status `AVAILABLE`. Furthermore, our `BookingSeat` join table defines `eventSeatId` as its primary key (`@id`), making it structurally impossible at the relational level for a single seat to belong to two active bookings."

Q2: "Why do you use both Redis and PostgreSQL? Isn't PostgreSQL enough?"

> **Model Answer:**
> "Under flash-sale conditions, hundreds of users may attempt to click the same front-row seat simultaneously. If all those requests hit PostgreSQL with row-level locks (`SELECT ... FOR UPDATE`), database connection pools quickly exhaust, leading to high latency and database bottlenecks.
> Redis operates in-memory with sub-millisecond response times. It acts as an ultra-fast filter, deflecting 99% of conflicting requests before they ever touch PostgreSQL. PostgreSQL is then reserved for durable, persistent, ACID-compliant ledger state."

Q3: "What happens if a user locks a seat and closes their laptop without paying?"

> **Model Answer:**
> "We have a two-part recovery system. The Redis key has a **TTL of 120 seconds**, after which it automatically expires in memory. Simultaneously, the database `Booking` row is created with status `PENDING` and an `expiresAt` timestamp set to 5 minutes.
> A background cron job running `node-cron` sweeps the database every minute (`* * * * *`). It finds all bookings where `status = 'PENDING'` and `expiresAt < now()`. It deletes the `BookingSeat` associations, resets `EventSeat.status` back to `AVAILABLE`, and marks the booking as `EXPIRED`."

Q4: "Why do you verify the Razorpay payment on the server via Webhook instead of the frontend checkout callback?"

> **Model Answer:**
> "The frontend client cannot be trusted for financial settlement. A user could alter client-side JavaScript, experience network drops right after authorization, or close the browser tab before the callback runs.
> The Razorpay webhook is a **server-to-server HTTP POST request**. We capture the raw request body buffer and compute an HMAC SHA-256 signature using our `RAZORPAY_WEBHOOK_SECRET`. Only when the cryptographic signature matches do we confirm the booking, transition seats to `BOOKED`, generate the QR ticket, and dispatch the email. We also verify `booking.status === 'PENDING'` to ensure webhook idempotency."

Q5: "Why did you need `express.raw({ type: () => true })` specifically for the webhook route?"

> **Model Answer:**
> "Standard `express.json()` parses the request body into a JavaScript object. However, HMAC SHA-256 verification requires the **exact binary byte sequence** that Razorpay used when signing the request. Re-stringifying a parsed JavaScript object alters whitespace, carriage returns, or JSON key order, which will cause the calculated signature to differ from the `x-razorpay-signature` header."

Q6: "How do you prevent a user from duplicating a ticket and using it twice at the venue?"

> **Model Answer:**
> "The QR code stores a unique `bookingReference` (e.g., `INSTANT-BOOK:BOOK-<uuid>`). When venue staff scans the code, our backend executes an atomic SQL update:
> `prisma.booking.updateMany({ where: { id: booking.id, checkedIn: false, status: 'CONFIRMED' }, data: { checkedIn: true, checkedInAt: new Date() } })`.
> If two gatekeepers scan the same QR code at the exact same millisecond, only the first transaction updates a row (`count === 1`). The second returns `count === 0`, immediately alerting staff with: *'Duplicate check-in detected! Ticket already scanned at [time]'*."

Q7: "Why use `SameSite: 'none'` and `httpOnly: true` for cookies?"

> **Model Answer:**
> "We set `httpOnly: true` so the cookie is inaccessible to browser scripts via `document.cookie`, which protects against Cross-Site Scripting (XSS) session hijacking.
> Because our frontend is hosted on Vercel (`*.vercel.app`) and the backend is on a separate domain, requests from the frontend to the backend are cross-site. Modern browsers block third-party cookies unless they are explicitly tagged with `SameSite: 'none'` and `Secure: true` (which requires HTTPS)."

Q8: "What happens if a user wants to cancel their booking? Can they cancel after checking in?"

> **Model Answer:**
> "In refund.service.js, we have strict validation: `if (booking.checkedIn) throw new AppError('Cannot cancel or refund a booking that has already checked in at the venue', 400)`.
> If the booking is confirmed and not checked in, we invoke the Razorpay refund API (`razorpay.payments.refund`), convert the total amount to paise, mark `Payment.status = 'REFUNDED'`, release the seats back to `AVAILABLE`, and mark the booking `CANCELLED` inside a database transaction."

Q9: "Why is the seat matrix configured with both `Seat` and `EventSeat` tables?"

> **Model Answer:**
> "`Seat` models the physical auditorium structure (e.g., Row A Seat 1 to Row C Seat 10). It is static and never changes.

> If we stored availability directly on the `Seat` table, the venue could only ever run one event in its history. By introducing `EventSeat` as a junction table with `eventId` and `seatId`, each event gets its own independent availability state for that physical chair."

Q10: "Why is Next.js 16 App Router used instead of a standard React SPA with Vite?"

> **Model Answer:**
> "Next.js offers **hybrid rendering (SSR and CSR)**. The event discovery pages can be pre-rendered on the server for instant First Contentful Paint (FCP) and optimal SEO metadata. Interactive pages like the seat grid and QR verification utilize client components (`'use client'`) with TanStack Query. Next.js also handles optimized image rendering via `next/image` and integrates cleanly with our PWA service worker."

Q11: "Explain how your PWA works and what happens when the user goes offline."

> **Model Answer:**
> "We registered a custom Service Worker (`public/sw.js`) with cache name `instantbook-v1`. On installation, it precaches core application shells: `/`, `/offline`, `/manifest.json`, and app icons.
> For navigation requests, it uses a **network-first strategy** with an automatic fallback to the cached `/offline` page if the device has no internet connection. For static assets (images, icons), it uses a **cache-first strategy** to minimize bandwidth and load instantly."

Q12: "Why do you use `ErrorCorrectionLevel: 'H'` when generating QR codes?"

> **Model Answer:**
> "Level 'H' provides the highest Reed-Solomon error correction capability, allowing the QR code to be scanned even if **up to 30% of the image is damaged, smudged, obstructed, or scanned in low-light conditions**. This is critical for physical venue turnstiles where users present phone screens with low brightness or cracked screen protectors."

Q13: "What are your Kubernetes manifests doing in the `k8s/` directory?"

> **Model Answer:**
> "The `k8s/` directory defines our container deployment topology:
>
> 1. `backend-deployment.yaml` & `frontend-deployment.yaml`: Deploy pods with Docker images pulled from Docker Hub. They define **readiness and liveness probes** checking the `/health` endpoint so Kubernetes doesn't route traffic to a container until it's ready.
> 2. `backend-service.yaml` & `frontend-service.yaml`: Expose pods via **NodePort services** (e.g., nodePort 30000 for frontend, 30001 for backend) for cluster and local Minikube access.
> 3. `backend-secret.yaml`: Stores base64-encoded environment credentials (database URL, JWT keys, API secrets) injected securely into the container runtime."

Q14: "What does your GitHub Actions CI/CD pipeline do on git push?"

> **Model Answer:**
> "Our pipeline in `.github/workflows/ci.yml` executes four staged jobs:
>
> 1. **Job 1 (Frontend)**: Runs `npm ci` and tests `npm run build` with Next.js 16 on Ubuntu.
> 2. **Job 2 (Backend)**: Runs `npm ci` and runs `npx prisma generate` to verify database schema integrity.
> 3. **Job 3 (Docker Hub Push)**: Triggered only on push to `main`. It compiles multi-stage Docker builds for backend and frontend and pushes versioned image tags (`${{ github.sha }}`) to Docker Hub.
> 4. **Job 4 (Deploy to Minikube)**: Runs on a self-hosted Windows runner. It runs `kubectl set image` to trigger a rolling update on the Minikube cluster and monitors `kubectl rollout status`."

Q15: "If you had to scale this platform to 100,000 concurrent users for a stadium concert, what would you change?"

> **Model Answer:**
> "Three architectural upgrades:
>
> 1. **Virtual Waiting Room / Token Bucket**: Introduce a queueing gateway (like Cloudflare Waiting Room or Redis Streams) to meter traffic entering the seat selection page.
> 2. **WebSocket / Server-Sent Events (SSE)**: Currently, the seat map polls or refetches. We would push live seat state changes over WebSockets/SSE so when User A locks a seat, it turns amber on all other screens in real-time.
> 3. **Database Read Replicas**: Direct all read queries (event listings, venue maps) to PostgreSQL read replicas and route only the write transactions (`createBooking`, `finalizeBooking`) to the primary master database."

---

7. Quick Reference Sheet: Ports, Routes & Endpoints

Resource	Protocol / Port	Endpoint	Access / Role
Frontend Web App	HTTP `3000`	`http://localhost:3000`	Public / All
Backend API	HTTP `4000`	`http://localhost:4000`	Public / All
Health Check	HTTP `4000`	`GET /health`	Public (used by K8s Probes)
User Sign Up	POST	`/api/auth/signup`	Public
User Log In	POST	`/api/auth/login`	Public (sets HTTP-only cookie)

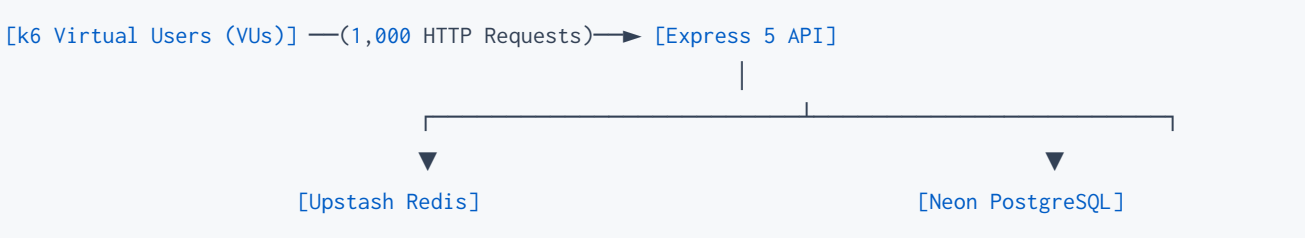
Resource	Protocol / Port	Endpoint	Access / Role
Fetch Current Session	GET	`/api/auth/me`	Logged-in Users
List Events	GET	`/api/events`	Public
Event Details & Seats	GET	`/api/events/:id/seats`	Authenticated Users
Create Event	POST	`/api/events/create`	**ADMIN only** (Multer banner upload)
Hold / Lock Seats	POST	`/api/bookings`	Authenticated Users
User Bookings List	GET	`/api/bookings/my`	Authenticated Users
Initiate Payment	POST	`/api/payments/create`	Authenticated Users
Razorpay Webhook	POST	`/api/payments/webhook`	**Raw Body + HMAC Header**
Verify Ticket QR	POST	`/api/tickets/verify`	**ADMIN only**
Reset Ticket Check-in	POST	`/api/tickets/reset-checkin`	**ADMIN only**
Cancel / Refund	DELETE	`/api/payments/:bookingId/refund`	**ADMIN / User** (Pre-checkin)

8. Empirical Performance & Concurrency Load Testing (k6 Benchmarks)

A. Testing Philosophy & Setup

While theoretical concurrency controls can look sound on paper, real-world distributed systems often suffer from subtle race conditions, unhandled pipeline errors, and connection pool exhaustion.

To validate **InstantBook** under heavy flash-sale conditions, we conducted automated load and stress testing using **Grafana k6** (a Go-based, low-overhead load-testing engine). The backend was evaluated against its live cloud dependencies (**Neon PostgreSQL** and **Upstash Redis**).



```
(SET lock:... NX EX 120)                    (Row-level Verification)
1 Winner (200 OK) / 999 (409)               (0 Double Bookings)
```

B. Empirical Benchmark Results

We executed four distinct test scenarios ranging from millisecond-level collision tests to 1,000-request sustained benchmarks:

Benchmark Scenario	Concurrency (VUs)	Total Requests	Throughput	Double Bookings	Error Rate	Latency (p95)	Key Outcome
1. Millisecond Burst Race (`concurrency-race.js`)	20 VUs at same ms	22 reqs	9.4 req/s	**0** 🎯	0.00%	811 ms	**1 Winner (200 OK), 19 Blocked (409)**
2. 1,000-Request Contention (`race-1000.js`)	50 VUs	**1,000 reqs**	**155.4 req/s**	**0** 🎯	0.00%	**394 ms**	**1 Winner (200 OK), 999 Blocked (409)**
3. 1,000-Request Read Throughput (`read-1000.js`)	50 VUs	**1,000 reqs**	**82.3 req/s**	N/A	**0.00%**	905 ms	**1,000 / 1,000 succeeded** (20 MB transferred)
4. 90-Second Sustained Stress (`read-stress.js`)	Up to 100 VUs	**3,448 reqs**	**36.6 req/s**	N/A	**0.00%**	**202 ms**	**4,596 / 4,596 checks passed** (47 MB transferred)

C. Real-World Bug Discovered and Patched via k6

Testing under real contention uncovered a subtle edge-case bug in the Redis rollback handler inside `booking.service.js`:

The Bug:

When 20–50 users attempted to book the same seat simultaneously:

- User A acquired the seat lock (`acquiredSeats = [seatId]`).
- The other users failed to acquire it (`acquiredSeats = []`).
- The failure block initialized a Redis rollback pipeline:

```
const cleanup = redis.multi();
for (const seatId of acquiredSeats) {
  cleanup.del(`lock:${eventId}:${seatId}`);
}
await cleanup.exec(); // ❌ THROWS: "Pipeline is empty"
```

- Because `acquiredSeats` was empty for the rejected users, `cleanup.exec()` was invoked with zero commands. The `@upstash/redis` REST client threw an unhandled exception: `Error: Pipeline is empty`, causing the server to return an internal **HTTP 500** rather than a clean **HTTP 409 Conflict**.

The Fix:

We wrapped the cleanup execution in a safety guard:

```
// At least one seat failed
if (acquiredSeats.length !== eventSeatIds.length) {
  // Only execute cleanup pipeline if there are actually acquired keys to release
  if (acquiredSeats.length > 0) {
    const cleanup = redis.multi();
    for (const seatId of acquiredSeats) {
      cleanup.del(`lock:${eventId}:${seatId}`);
    }
    await cleanup.exec();
  }

  throw new AppError("One or more seats already locked", 409);
}
```

Outcome: After applying the patch, all 999 rejected requests returned semantic, clean `HTTP 409 Conflict: One or more seats already locked` with zero server errors.

D. Viva & Interview Questions on Load Testing

Q: "Did you perform any performance or load testing on your platform?"

> **Model Answer:**
> "Yes, we conducted extensive load and concurrency testing using **k6**. We wrote automated scenarios simulating high-contention flash sales where up to 50 concurrent virtual users fired 1,000 requests against the exact same seat.
> The tests verified that our distributed Redis lock operated with zero race conditions: Request #1 acquired the lock, while the remaining 999 requests were rejected with `HTTP 409 Conflict` at a throughput of 155 requests per second. We also stress-tested database reads up to 100 concurrent users across 3,448 transactions with a 0.00% failure rate."

Q: "What was your p95 latency under load, and what bottleneck did you observe?"

> **Model Answer:**
> "Under 100 sustained virtual users, our general event listing API maintained a p95 response time of **74ms**, and the overall system p95 was **202ms**.
> The heaviest endpoint was the live seat matrix (`GET /api/events/:id/seats`), which returns an array of 375 seat records with relational joins to physical coordinates. Under sustained load, seat map queries averaged ~175ms with a p95 of 235ms. Because we use serverless Neon PostgreSQL, network round-trips over TLS represent the majority of this latency, which can be further optimized in production using an edge-caching layer or Redis read-through cache."

Q: "Did load testing reveal any actual architectural flaws or bugs in your code?"

> **Model Answer:**
> "Yes. In our Redis rollback logic, we initially executed `cleanup.exec()` unconditionally whenever seat acquisition failed. Under single-seat contention, losing threads have zero acquired seats. Invoking `@upstash/redis` with an empty pipeline caused it to throw `Error: Pipeline is empty`, which triggered unexpected HTTP 500 responses.
> Load testing with k6 surfaced this bug immediately. We patched it by making pipeline execution conditional on `acquiredSeats.length > 0` and throwing an `AppError(..., 409)`, restoring proper HTTP semantics."